

React Long-Term Project Roadmap

A single evolving Movie Discovery / Watchlist application used as the baseline project for Sessions 10–18.

Teaching Strategy

The project should not be rebuilt from scratch for every topic. Instead, students continuously improve the same application. Each new React concept is introduced because the growing application has created a problem that the concept can solve.

Teaching pattern: Identify the application's problem → let students consider how they would solve it → introduce the React concept → implement it in the existing project.

Session	Problem / Feature	React Topic Applied	Project Evolution
10	The app needs real movie data instead of hard-coded movies.	Fetching Data & APIs; useEffect; loading/error states	Movie discovery page with API-powered movie cards.
10	The UI needs to display many movies returned by the API.	Lists & Keys; component rendering	Reusable MovieCard and MovieGrid components.
10	Users need to search for movies.	Events; controlled inputs; useState	Search bar connected to the movie API.
10	The application needs to handle slow requests and failed requests.	Conditional Rendering; loading/error/empty states	Loading indicator, error message, and no-results state.
11	The application is becoming larger than one screen.	React Router	Home, Search, Movie Details, and Watchlist routes.
11	Every movie needs its own URL.	Dynamic Routes; useParams	Routes such as /movies/:id for individual movie details.
11	Users need to move around the application without full page reloads.	Link; useNavigate	Navigation bar and programmatic navigation.
12	Several unrelated components need access to the same theme.	Context API	Global Theme Context with light/dark mode.
12	Passing shared data through many components creates prop drilling.	Context API; Providers; useContext	Shared application-level state without unnecessary prop passing.
13	The search/filter experience now requires multiple fields and validation.	Formik + Yup; advanced forms	Advanced movie search/filter form with validation.
13	Users may need to submit structured information such as reviews.	Form submission; validation; error handling	Movie review/rating form on the details page.
14	The watchlist now has several related state transitions.	useReducer	Centralized watchlist reducer.
14	Users need to add, remove, toggle, and clear saved movies.	Reducer actions and dispatch	ADD_MOVIE, REMOVE_MOVIE, TOGGLE_MOVIE, CLEAR_WATCHLIST.
14	The watchlist must be available from different parts of the application.	Context + useReducer	Global Watchlist Provider backed by a reducer.
15	The application has important behavior that could break during changes.	Jest + React Testing Library	Tests for important UI behavior and user interactions.
15	Movie data can be loading, successful, empty, or failed.	Component/API testing	Tests for loading, success, empty, and error states.
15	Watchlist behavior must remain reliable.	Interaction testing	Tests for adding/removing movies and updating the watchlist.
16	The application needs additional core features before it feels complete.	Integration of previously learned concepts	Complete remaining core features such as favorites, filtering, pagination, or profile functionality.

Session	Problem / Feature	React Topic Applied	Project Evolution
16	The codebase is growing and needs better organization.	Component architecture; reusable components	Organize components, pages, contexts, reducers, services, hooks, and utilities according to project needs.
17	The application works but does not yet feel polished.	Styling; responsive UI	Consistent layouts, responsive screens, reusable UI patterns.
17	Users need clear feedback when actions are pending, empty, or unsuccessful.	Conditional rendering; UI states	Skeleton/loading states, empty states, error states, and useful feedback.
18	The application needs to be accessible to real users.	Build/deployment workflow	Production build and deployment to a platform such as Vercel or Netlify.
18	Students need to demonstrate what they built.	Project presentation; course wrap-up	Live project demo, explanation of architecture/features, Q&A.;

Project Evolution by Version

Version 1 — Session 10: Movie Discovery

One-page application that fetches movies from an API, displays reusable movie cards, supports search, and handles loading, errors, and empty results.

Version 2 — Session 11: Multi-page Movie Platform

Add React Router, navigation, movie detail pages, dynamic movie IDs, search pages, and a watchlist route.

Version 3 — Session 12: Shared Application State

Add Context for a global theme and remove unnecessary prop drilling.

Version 4 — Session 13: Advanced Search & Reviews

Add a more complex validated form for filtering/searching and optionally a movie review/rating form.

Version 5 — Session 14: State Architecture

Replace increasingly complicated watchlist state with useReducer, then expose it globally through Context.

Version 6 — Session 15: Tested Application

Add automated tests around rendering, API states, user interactions, and watchlist behavior.

Version 7 — Sessions 16–18: Final Production Project

Complete remaining features, improve architecture and responsive UI, polish UX, deploy, and present the finished application.

Suggested Final Application Structure

The exact folder structure should emerge as the project grows rather than being imposed on students from the beginning.

```
src/  
■■■ components/  
■■■ pages/  
■■■ context/  
■■■ reducers/  
■■■ hooks/  
■■■ services/  
■■■ utils/  
■■■ assets/  
■■■ App.jsx  
■■■ main.jsx
```

Core Teaching Principle

Do not teach the API first and search for a use for it afterward. Let the application's growth create the need for the API. The same principle should be used for Router, Context, Formik/Yup, useReducer, and testing.

The goal is for students to finish the course understanding not only *how* to use React features, but *why* a feature becomes useful when an application reaches a particular level of complexity.